

Python Multiprocessing Pool

Complete Learning Guide

Master all 7 pool methods with detailed explanations, comparisons, real-world examples, and best practices

Generated: September 26, 2026

Table of Contents

1. Introduction
2. Understanding Multiprocessing
3. Method Overview
4. Blocking Methods (map, starmap, apply)
5. Non-Blocking Methods (async, imap)
6. Complete Comparison
7. Speed & Memory Analysis
8. Real-World Scenarios
9. Decision Guide
10. Best Practices
11. FAQ

1. Introduction

Python's multiprocessing module allows you to run code in parallel using multiple processes. The Pool object is the most convenient way to distribute work across multiple workers. However, with 7 different methods available, choosing the right one is crucial for performance and code clarity. This guide explains all 7 pool methods in depth: when to use them, their trade-offs, and real-world examples. By the end, you'll know exactly which method to use for any parallel processing task.

2. Understanding Multiprocessing

What is Multiprocessing?

Multiprocessing runs tasks in separate processes, each with its own Python interpreter. Unlike threading, multiprocessing truly executes code in parallel on multiple CPU cores. This is perfect for CPU-bound tasks like:

- Image processing and computer vision
- Data analysis and calculations
- Machine learning model inference
- Scientific simulations
- Batch data processing

Pool vs Process

A Pool manages a group of worker processes. Instead of creating processes manually, the Pool handles:

- Creating worker processes
- Distributing tasks to workers
- Collecting results
- Cleaning up when done

This makes Pool the easiest way to parallelize work across multiple cores.

3. Method Overview - All 7 Methods

Method	Blocks?	Input	Speed	Memory	Best For
apply()	YES	1 value	■ SLOW	Low	■ NEVER USE
map()	YES	List	■ FAST	High	✓ DEFAULT
starmap()	YES	Tuples	■ FAST	High	Multi args
apply_async()	NO	1 value	■ INSTANT	Low	1 bg task
map_async()	NO	List	■ INSTANT	High	Many bg
imap()	NO	List	■ FAST	■ LOW	Large data
imap_unordered()	NO	List	■ FASTEST	■ LOW	BEST

4. Blocking Methods

4.1 pool.apply() - ■ AVOID

What it does: Processes ONE task at a time, waits for it to complete before moving to the next.

Blocking: YES - Main program waits for each task

Number of workers: Only 1 worker is active at a time (others sit idle)

Returns: Single value

Example:

```
with Pool(processes=3) as pool:
    for n in [1,2,3,4,5]:
        result = pool.apply(square, (n,)) # Waits here
```

Why avoid it? With 5 items and 3 workers, only 1 worker runs at a time. Other workers are wasted! This is 3x slower than using map().

Use case: NONE - Use map() or other methods instead.

4.2 pool.map() - ✓ DEFAULT (Use this!)

What it does: Processes MULTIPLE tasks in parallel, waits for ALL to complete.

Blocking: YES - Main program waits for all results

Number of workers: ALL workers run in parallel

Returns: List of results (same order as input)

Example:

```
with Pool(processes=3) as pool:
    results = pool.map(square, [1,2,3,4,5])
    # Returns: [1, 4, 9, 16, 25]
```

Why use it? Simple, fast, and results come in predictable order. All workers run in parallel - with 5 items and 3 workers, it takes 2 rounds instead of 5!

Use case: ✓ DEFAULT CHOICE - Process multiple items, can wait for results. Works 95% of the time.

4.3 pool.starmap() - Multiple Arguments

What it does: Like map() but for functions that need MULTIPLE arguments.

Blocking: YES - Main program waits for all results

Number of workers: ALL workers run in parallel

Returns: List of results

Example:

```
def multiply(a, b):  
    return a * b
```

```
with Pool(processes=3) as pool:  
    results = pool.starmap(multiply, [(2,3), (4,5), (6,7)])  
    # Returns: [6, 20, 42]
```

Why use it? Automatically unpacks tuples into function arguments. Same speed as `map()`, just handles multiple arguments.

Use case: ✓ When your function needs multiple parameters (not single list).

5. Non-Blocking Methods

5.1 pool.apply_async() - Single Background Task

What it does: Starts 1 task but DOESN'T WAIT. Main program continues immediately.

Blocking: NO - Returns instantly (task runs in background)

Number of workers: 1 worker processes the task

Returns: AsyncResult object (call .get() later)

Example:

```
with Pool(processes=2) as pool:
    async_result = pool.apply_async(square, (10,))
    print("Task started!")
    # Main program continues here
    result = async_result.get() # Wait for result
```

AsyncResult methods:

- .get() - Get result (blocks if not ready)
- .ready() - Check if done (non-blocking)
- .get(timeout=5) - Wait max 5 seconds

Use case: ✓ Single heavy background task (download file, heavy computation) while main program stays responsive.

5.2 pool.map_async() - Multiple Background Tasks

What it does: Starts MULTIPLE tasks but DOESN'T WAIT. Main program continues immediately.

Blocking: NO - Returns instantly (tasks run in background)

Number of workers: ALL workers run in parallel

Returns: AsyncResult object (call .get() for list)

Example:

```
with Pool(processes=3) as pool:
    async_result = pool.map_async(square, [1,2,3,4,5])
    print("All tasks started!")
    # Main program continues
    results = async_result.get() # Wait for all
```

Use case: ✓ Process many items in background while main program (UI server, web app) stays responsive.

5.3 pool.imap() - Memory-Efficient Streaming (Ordered)

What it does: Returns a GENERATOR. Processes results one-by-one in INPUT ORDER as workers finish them.

Blocking: NO - Returns generator immediately

Number of workers: ALL workers run in parallel

Memory: ■ VERY LOW - Only 1 result in memory at a time

Returns: Generator (not a list)

Example:

```
with Pool(processes=3) as pool:
    for result in pool.imap(square, [1,2,3,4,5]):
        print(f"Got: {result}") # Process immediately
        save_to_disk(result)
```

Why use it? Processing 1 million items? Don't load all results into memory! imap yields results one-by-one, keeping memory usage low. Results come in INPUT ORDER.

Use case: ✓ Large datasets (1000+ items) where memory matters and results must be in order.

5.4 pool.imap_unordered() - Fastest Memory-Efficient

What it does: Like imap() but results come in COMPLETION ORDER (whichever finishes first).

Blocking: NO - Returns generator immediately

Number of workers: ALL workers run in parallel

Memory: ■ VERY LOW - Only 1 result in memory at a time

Returns: Generator (results in any order)

Speed: ■ FASTEST option overall

Example:

```
with Pool(processes=3) as pool:
    for result in pool.imap_unordered(square, [1,2,3,4,5]):
        print(f"Got: {result}") # Might print: 1, 25, 9, 4, 16
```

Why use it? Fastest option! Results come as soon as they're ready, not in input order. Perfect for batch processing where you don't care about order.

Use case: ✓ HUGE datasets (millions of items), speed critical, order doesn't matter.

6. Complete Comparison Table

Aspect	apply()	map()	starmap()	apply_async()	map_async()	imap()	imap_unordered()
Blocks	YES	YES	YES	NO	NO	NO	NO
Tasks	1 only	Multiple	Multiple	1 only	Multiple	Multiple	Multiple
Speed	■	■	■	■	■	■	■ BEST
Memory	Low	High	High	Low	High	■ Low	■ Low
Result Order	N/A	Input	Input	N/A	Input	Input	Completion
Use Case	■ NEVER	✓ DEFAULT	Multi args	1 BG task	Many BG	Large data	HUGE data

7. Speed & Memory Analysis

Processing 12 items with 3 workers (each takes 0.5s)

Sequential (no parallelization): 6.0s (1 + 1 + 1 + 1 + 1 + 1)

apply(): 6.0s (only 1 worker runs!)

map(), starmap(): 2.0s (3 workers run parallel - 4 rounds)

apply_async(): ~2.0s (returns instantly, background)

map_async(): ~2.0s (returns instantly, background)

imap(): 2.0s (same speed, streams results)

imap_unordered(): 2.0s (same speed, fastest delivery)

Key insight: apply() is 3x slower because only 1 worker runs at a time!

Memory Usage - Processing 1,000,000 items

map(): Loads all 1,000,000 results in memory (~40 MB) ■■

starmap(): Loads all 1,000,000 results in memory (~40 MB) ■■

apply_async(): Single result in memory ✓

map_async(): Queues all items, loads all results (~40 MB) ■■

imap(): Only 1 result in memory at a time ■ BEST

imap_unordered(): Only 1 result in memory at a time ■ BEST

Rule of thumb: Use imap() or imap_unordered() for huge datasets to avoid memory issues!

8. Real-World Scenarios

Scenario 1: Process 10 Product Images

Problem: Resize 10 product images for a web gallery

Best method: `pool.map()`

Why: Only 10 items, don't need memory optimization, can wait for all results

Code: `results = pool.map(resize_image, images)`

Scenario 2: Download 1000 Files

Problem: Download 1000 files from URLs, order doesn't matter

Best method: `pool.imap_unordered()`

Why: Large dataset, memory efficient, speed critical, order irrelevant

Code: `for file in pool.imap_unordered(download_file, urls):
 save_to_disk(file)`

Scenario 3: Web Server + Background Processing

Problem: Server receives request to process 100 images, must respond immediately

Best method: `pool.map_async()` or `pool.apply_async()`

Why: Non-blocking, main server stays responsive, processing happens in background

Code: `async_result = pool.map_async(process, images)
return {"status": "processing"} # Respond immediately`

Scenario 4: ML Model - Process Sensor Data

Problem: 1 million sensor readings, must process in order (time-series)

Best method: `pool.imap()`

Why: Large dataset, results must be ordered, memory efficient

Code: `for reading in pool.imap(process_sensor, data):
 save_to_database(reading)`

9. Decision Guide - How to Choose

Step 1: Do you have multiple items to process?

YES: Continue to Step 2

NO: You don't need multiprocessing

Step 2: Can your program WAIT for all results?

YES: Use `pool.map()` (go to Step 3)

NO: Use `pool.map_async()` or `pool.apply_async()` (non-blocking)

Step 3: Do you have < 1000 items?

YES: Use `pool.map()` ✓ (DONE - simplest choice)

NO: Continue to Step 4

Step 4: Do you need results in specific order?

YES: Use `pool.imap()` (memory efficient, ordered)

NO: Use `pool.imap_unordered()` (fastest + memory efficient)

Quick Decision Matrix

Your Situation	Use This
Multiple items, can wait, simple	<code>pool.map()</code>
Multiple items, function needs 2+ args	<code>pool.starmap()</code>
1 heavy background task	<code>pool.apply_async()</code>
Many background tasks	<code>pool.map_async()</code>
1000+ items, ordered	<code>pool.imap()</code>
1000+ items, speed critical	<code>pool.imap_unordered()</code>
Still unsure	<code>pool.map()</code> ✓

10. Best Practices

✓ DO:

1. **Use "with" statement:** Automatically closes pool and cleans up workers

```
with Pool() as pool:  
    results = pool.map(func, items)
```

2. **Set processes parameter:** Usually number of CPU cores

```
import os  
processes = os.cpu_count() # 4, 8, 16 depending on CPU
```

3. **Keep functions simple:** Worker processes can't access main process state

4. **Use imap for large datasets:** Saves memory, processes results immediately

5. **Set timeout for blocking calls:** Prevents hanging forever

```
result = async_result.get(timeout=30)
```

6. **Use pool.map() by default:** Simplest, works for 95% of cases

X DON'T:

1. **Don't use pool.apply():** Only 1 worker runs, extremely slow

2. **Don't assume shared state:** Each worker has separate memory

X Won't work: `x = 0` # Can't modify from worker

3. **Don't use lambda functions:** Can't be pickled for multiprocessing

X Won't work: `pool.map(lambda x: x*2, items)`

4. **Don't forget to close pool:** Use "with" statement

5. **Don't use for fast functions:** Overhead isn't worth it for microsecond operations

6. **Don't load all results for huge datasets:** Use `imap/imap_unordered`

11. Frequently Asked Questions

Q: How many workers should I use?

A: Usually `os.cpu_count()` (your CPU core count). For I/O-bound tasks, can use more.

Q: Why is `apply()` so slow?

A: Only 1 worker processes tasks at a time. Others sit idle. Never use it!

Q: What's the difference between `map()` and `imap()`?

A: `map()` blocks until all done, loads all results. `imap()` streams results one-by-one.

Q: Should I use async methods?

A: Only if main program must stay responsive (web servers, UIs). For batch jobs, use `map()`.

Q: Can workers share variables?

A: No. Each worker has separate memory. Use return values to communicate.

Q: Why can't I use lambda functions?

A: Lambda functions can't be serialized (pickled) to send to worker processes.

Q: What's the memory limit for results?

A: Python lists load all results into RAM. With `imap`, only 1 result in memory at a time.

Q: Should I catch exceptions in workers?

A: Yes. Use `try/except` in your function. Exceptions propagate when you call `.get()`

Summary & Quick Reference

The 7 Pool Methods at a Glance:

1. **pool.apply()** - ■ NEVER USE (only 1 worker)
2. **pool.map()** - ✓ DEFAULT (95% of cases)
3. **pool.starmap()** - Multiple arguments
4. **pool.apply_async()** - 1 background task
5. **pool.map_async()** - Many background tasks
6. **pool.imap()** - Large data, ordered
7. **pool.imap_unordered()** - Huge data, fastest

Your Next Step:

1. Identify how many items you need to process
2. Use the Decision Guide (page 13)
3. Follow the examples provided
4. Run the Python files for hands-on practice

Remember:

- Multiprocessing is for CPU-bound tasks (calculations, image processing, ML)
- Threading is for I/O-bound tasks (network, file I/O)
- Always use "with" statement for automatic cleanup
- `pool.map()` is your default answer - use it first
- When in doubt, profile and test with real data

Good luck with your parallel processing! ■